CS2601 Linear and Convex Optimization
Homework 12 Solution

Due: 2021.12.25

For this assignment, you should submit a **single** pdf file as well as your source code (.py or .ipynb files). The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions. Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

1. Let's revisit Problem 2 of Homework 11.

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) = e^{x_1} + e^{2x_2} + e^{2x_3} \\ \text{s.t.} \quad & x_1 + x_2 + x_3 = 1 \end{aligned} \tag{1}$$

(a). Find the closed-form expression for the Newton direction at a feasible $\boldsymbol{x}$ by solving the KKT system.

(b). Implement the constrained Newton's method on slide 12 of §12 in the `newton_eq` function of `newton.py`. The functions `numpy.block` and `numpy.linalg.solve` might be useful. Use your implementation to solve (1) with the initial point $\boldsymbol{x}_0 = (0, 1, 0)^T$. Show the output.

Solution. (a). The KKT system is

$$\begin{bmatrix} e^{x_1} & 0 & 0 & 1 \\ 0 & 4e^{2x_2} & 0 & 1 \\ 0 & 0 & 4e^{2x_3} & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \lambda \end{bmatrix} = \begin{bmatrix} -e^{x_1} \\ -2e^{2x_2} \\ -2e^{2x_3} \\ 0 \end{bmatrix}$$

The Newton's direction is $\boldsymbol{d} = (d_1, d_2, d_3)^T$, where

$$\begin{aligned} d_1 &= -1 + \frac{2e^{-x_1}}{e^{-x_1} + \frac{1}{4}e^{-2x_2} + \frac{1}{4}e^{-2x_3}} \\ d_2 &= -\frac{1}{2} + \frac{\frac{1}{2}e^{-2x_2}}{e^{-x_1} + \frac{1}{4}e^{-2x_2} + \frac{1}{4}e^{-2x_3}} \\ d_3 &= -\frac{1}{2} + \frac{\frac{1}{2}e^{-2x_3}}{e^{-x_1} + \frac{1}{4}e^{-2x_2} + \frac{1}{4}e^{-2x_3}} \\ \lambda &= \frac{-2}{e^{-x_1} + \frac{1}{4}e^{-2x_2} + \frac{1}{4}e^{-2x_3}} \end{aligned}$$

(b). The output is

```
iteration 0: [0. 1. 0.]
iteration 1: [ 0.55783402  0.55270748 -0.1105415 ]
iteration 2: [0.74171111 0.22388047 0.03440841]
iteration 3: [0.83735858 0.09139269 0.07124873]
iteration 4: [0.8464719  0.07685719 0.07667091]
iteration 5: [0.84657358 0.07671322 0.0767132 ]
iteration 6: [0.84657359 0.0767132  0.0767132 ]
optimal value: 4.663287963194248
```

□

2. Consider the LP in standard form,

$$\begin{aligned} \min_{\boldsymbol{x}\in\mathbb{R}^n} \quad & \boldsymbol{c}^T\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ & \boldsymbol{x} \geq \boldsymbol{0} \end{aligned}$$

We are going to solve it using the barrier method.

(a). Write down the approximating equality constrained problem.

(b). Write down the gradient and Hessian matrix of the objective function you find in (a).

(c). Implement the barrier method for solving a generic LP in standard form with a given feasible initial point. Complete the functions `centering_step` and `barrier` in `LP.py`. For the centering step, i.e. line 3 on slide 13 of §12, you can use your implementation of constrained Newton's method in Problem 1(b) by defining the penalized objective function and its derivatives inside `centering_step`.

(d). Consider the LP on slide 6 of §13,

$$\begin{aligned} \min_{\boldsymbol{x}\in\mathbb{R}^2} \quad & -x_1 - 3x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 6 \\ & -x_1 + 2x_2 \leq 8 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Convert it to standard form and then use your implementation in (c) to solve it. You can find a feasible initial point by setting $x_1 = 2, x_2 = 1$ and the slack variables appropriately. Show the output. Note `p2.py` also plots the projection of the iterates onto the x_1, x_2 coordinates.

Proof. (a).

$$\begin{aligned} \min_{\boldsymbol{x}\in\mathbb{R}^n} \quad & F(\boldsymbol{x}) = \boldsymbol{c}^T\boldsymbol{x} - \frac{1}{t}\sum_{i=1}^{n}\log x_i \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \end{aligned}$$

(b).

$$\nabla F(\boldsymbol{x}) = \boldsymbol{c} - \frac{1}{t}\left(\frac{1}{x_1}, \ldots, \frac{1}{x_n}\right)^T$$

$$\nabla^2 F(\boldsymbol{x}) = \frac{1}{t}\operatorname{diag}\left\{\frac{1}{x_1^2}, \ldots, \frac{1}{x_n^2}\right\}$$

(c). A possible implementation:

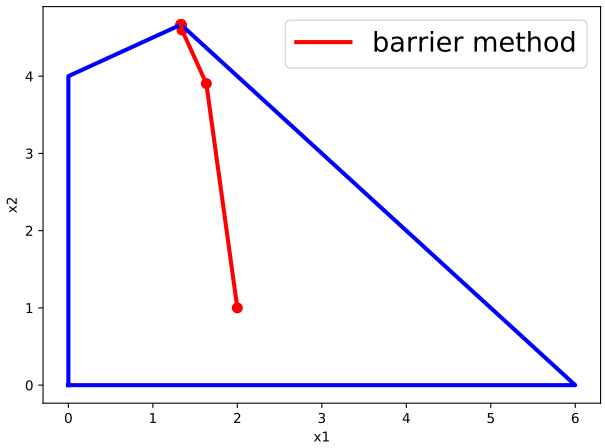
```
def centering_step(c, A, b, x0, t):
    F = lambda x:  c@x - 1/t * np.sum(np.log(np.maximum(x,0)))
    Fp = lambda x: c -  1 / (t * x)
    Fpp = lambda x: 1/t * np.diag(1/x**2)

    x_traces, _, _ = nt.newton_eq(F, Fp, Fpp, x0, A, b)
    return x_traces[-1]

def barrier(c, A, b, x0, tol=1e-8, t0=1, rho=10):
    t = t0
    x = np.array(x0)
    x_traces = [np.array(x0)]
    tc = len(b) / tol
    while t < tc:
        x = centering_step(c, A, b, x, t)
        x_traces.append(np.array(x))
        t *= rho

    return x_traces
```

(d).

$$\begin{aligned} \min_{\boldsymbol{x}\in\mathbb{R}^4} \quad & -x_1 - 3x_2 \\ \text{s.t.} \quad & x_1 + x_2 + x_3 = 6 \\ & -x_1 + 2x_2 + x_4 = 8 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{aligned}$$

The output is

```
iteration 0: [2. 1. 3. 8.]
iteration 1: [2.36022335 2.54500744 1.09476921 5.27020847]
iteration 2: [1.39480438 4.42512225 0.18007338 0.54455988]
iteration 3: [1.33696633 4.64325926 0.0197744  0.05044781]
iteration 4: [1.33366968e+00 4.66433261e+00 1.99771722e-03 5.00446578e-03]
iteration 5: [1.33336670e+00 4.66643333e+00 1.99977146e-04 5.00044644e-04]
iteration 6: [1.33333667e+00 4.66664333e+00 1.99997712e-05 5.00004459e-05]
iteration 7: [1.33333367e+00 4.66666433e+00 1.99999769e-06 5.00000441e-06]
iteration 8: [1.33333337e+00 4.66666643e+00 1.99979886e-07 4.99949820e-07]
iteration 9: [1.33333334e+00 4.66666664e+00 1.97990997e-08 4.94977523e-08]
optimal value: -15.333333267336327
```



□

3. Consider the LP on slide 6 of §13; see also Problem 2(d).

(a). Find the dual LP in the standard form, i.e. with all four dual variables.

(b). Find the symmetric dual LP, i.e. without the dual variables for the primal nonnegativity constraints.

(c). Solve the dual LP in (b) graphically. Compare the dual optimal value and primal optimal value. Note that the primal optimal solution is given on slide 15 of §5 part 1.

(d). Solve the dual LP in (a) using your implementation in Problem 2(c). Note you need to convert the maximization problem into a minimization problem. You can use the feasible initial point $\boldsymbol{\mu}_0 = (4, 1, 2, 3)^T$. Show the dual optimal solution and dual optimal value.

Proof. (a). The dual LP is

$$\begin{aligned} \max_{\boldsymbol{\mu}\in\mathbb{R}^4} \quad & -6\mu_1 - 8\mu_2 \\ \text{s.t.} \quad & -\mu_1 + \mu_2 + \mu_3 = -1 \\ & -\mu_1 - 2\mu_2 + \mu_4 = -3 \\ & \mu_1, \mu_2, \mu_3, \mu_4 \geq 0 \end{aligned}$$

(b). The symmetric dual LP is

$$\begin{aligned} \max_{\boldsymbol{\mu}\in\mathbb{R}^2} \quad & -6\mu_1 - 8\mu_2 \\ \text{s.t.} \quad & -\mu_1 + \mu_2 \leq -1 \\ & -\mu_1 - 2\mu_2 \leq -3 \\ & \mu_1, \mu_2 \geq 0 \end{aligned}$$

(c). The dual optimal solution is $\mu_1^* = \frac{5}{3}$, $\mu_2^* = \frac{2}{3}$. The dual optimal value is $-\frac{46}{3}$, which is the same as the primal optimal value.

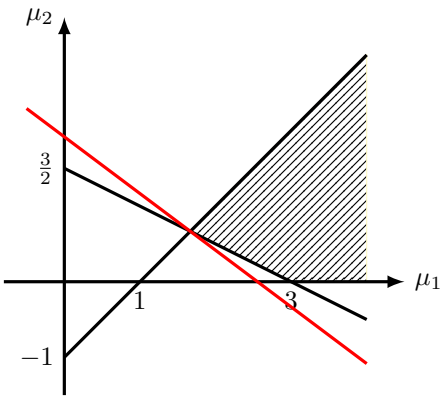


Figure 1: Dual LP

(d). The dual optimal solution is $(\frac{5}{3}, \frac{2}{3}, 0, 0)^T$ and the optimal solution is $-\frac{46}{3}$.

```
iteration 0: [4. 1. 2. 3.]
iteration 1: [2.1602764  0.54793489 0.61234151 0.25614619]
iteration 2: [1.72344886 0.64915465 0.0742942  0.02175816]
iteration 3: [1.67237818 0.66488395 0.00749423 0.00214608]
iteration 4: [1.66723807e+00 6.66488125e-01 7.49943598e-04 2.14317864e-04]
iteration 5: [1.66672381e+00 6.66648810e-01 7.49994366e-05 2.14288927e-05]
iteration 6: [1.66667238e+00 6.66664881e-01 7.49961753e-06 2.14275278e-06]
iteration 7: [1.66666724e+00 6.66666488e-01 7.49924434e-07 2.14264174e-07]
iteration 8: [1.66666672e+00 6.66666649e-01 7.42466209e-08 2.12133217e-08]
iteration 9: [1.66666667e+00 6.66666665e-01 6.66555703e-09 1.90444503e-09]
dual optimal value: -15.333333351108152
```

□